Independent Package Assessment

`@fm-budget-control/fm-budget-control-core`

Date: 2026-08-07 · **Revision assessed:** `main` @ `3252839` · **Published** `latest`: `2.0.0`

1. Executive summary and publication-readiness conclusion

The package is a small, zero-runtime-dependency, ESM-only TypeScript library of hexagonal-architecture domain primitives and application ports. Its **internal engineering quality is genuinely high**: 422 tests at 100% coverage on every instrumented file, immutable frozen value objects, deliberate secret redaction on `Password`, UTC-safe date handling that explicitly rejects JS `Date` auto-correction, integer-minor-unit money with overflow and currency-mismatch guards, type-level tests, SHA-pinned GitHub Actions, npm Trusted Publishing with a verified SLSA provenance attestation, and a release job gated behind a `production` environment approval.

That quality is undermined by three product-level defects that are **confirmed, not theoretical**:

1. **A breaking public-API change was published as a minor release.** `1.5.0` to `1.6.0` removed four methods from the two exported port interfaces and changed a third. This was verified against the registry tarballs, and the root cause reproduced: `@semantic-release/commit-analyzer`'s default `angular` preset does not parse the `!` breaking marker at all — `feat!:` commits are scored `null`, contributing nothing to version selection.
2. **The package ships no documentation whatsoever.** `readme.md` is 0 bytes and is packed into the tarball; `CHANGELOG.md` is excluded by `files` and is stale at `1.0.0` in git while nine versions are published. A package that has already undergone two breaking rewrites offers consumers no migration path.
3. **The "domain primitives" the package is named for are unreachable.** `Id`, `Money`, `CurrencyCode`, `IsoDate`, `IsoDateTime`, `Email`, `FullName`, `Password`, and `User` are all shipped in the tarball but absent from the `exports` map. 55 of 95 published files cannot be imported by any consumer.

Conclusion: Not ready as an independently maintained product. It functions correctly for its single known consumer, and nothing here is a security compromise — but the versioning contract is demonstrably broken and unfixed in configuration, and the consumer-facing surface is undocumented.

2. Scope, limitations, assumptions, and commands executed

Scope

Read-only assessment of the working tree at `main` @ `3252839`, plus the published registry artifacts for `1.5.0`, `1.6.0`, and `2.0.0`, plus the downstream consumer at `fm-budget-control-firebase/functions`.

Assumptions

The context placeholders in the request were unfilled; the following were derived from evidence.

Item	Value	Source
Package name	`@fm-budget-control/fm-budget-control-core`	`package.json:2`
Published state	**Public**, `latest = 2.0.0`, 9 versions, created 2026-06-15	`npm view`
Purpose	"Hexagonal architecture domain primitives"	`package.json:4`
Intended consumers	Assumed the `fm-budget-control` Firebase Functions backend; **only known consumer**	consumer `package.json` deps `^2.0.0`
Supported Node	**Undeclared.** `.nvmrc` = 24, CI = 24, consumer `engines.node` = 24	`.nvmrc`, `quality-gate.yml:35`
Supported TypeScript	**Undeclared.** Built with `typescript ^6.0.3`; verified working on consumer TS 5.9	`package.json:44`
Browsers / bundlers	**Not applicable** — assumed Node-only (Firebase Functions)	inferred
Compatibility commitments	**None stated anywhere**	no README / SECURITY / CONTRIBUTING

Limitations

- No network calls beyond `npm view` / `npm pack` of already-published versions and one `typescript` install into an isolated scratch consumer. No dependency was installed into, and nothing was modified in, the package repo.
- GitHub branch and tag protection rules, npm 2FA settings, the `production` environment's reviewer list, and the Trusted Publisher configuration **could not be verified** (require repo/org admin API access). Recorded as open questions.
- No runtime behaviour of the deployed Firebase Functions was exercised.
- No mutation testing or fuzzing was performed; correctness conclusions rest on reading plus the existing suite.

Commands executed

```
npm run typecheck | lint | test:types | test | test:coverage | build   (all green)
npm pack (local) ; tar -tzf ; npm pack --dry-run --json
npm view @fm-budget-control/fm-budget-control-core --json
npm pack @fm-budget-control/fm-budget-control-core@{1.5.0,1.6.0,2.0.0}
npm install <local tarball> into a clean ESM consumer; import/require probes
tsc across moduleResolution = nodenext | node16 | bundler | node
npm audit ; npm audit --omit=dev ; npm ls undici
git log/tag/show ; node -e "analyzeCommits(...)" (semantic-release preset repro)
```

All checks pass: typecheck clean, ESLint clean, type specs pass, **422/422 tests**, **100% statements / branches / functions / lines**, `publint` reports "All good!".

---

3. Package architecture and public API summary

```
src/
├── kernel/                          # shared, business-agnostic
│   ├── application/ports/           → HmacIdDeriverPort              [EXPORTED as ./kernel/ports]
│   └── domain/value-objects/        → Id, Money, CurrencyCode,
│                                      IsoDate, IsoDateTime           [NOT EXPORTED]
└── user/                            # bounded context
    ├── application/
    │   ├── ports/                   → UserRepositoryPort,
    │   │                              AuthProviderPort               [EXPORTED as ./user/ports]
    │   └── use-cases/register-user/ → RegisterUserUseCase            [EXPORTED as ./user/application]
    └── domain/                      → User, Email, FullName,
                                       Password, UserId, errors       [NOT EXPORTED]
```

Dependency direction is correct and enforced by structure: `user/domain` depends on `kernel/domain`; `user/application` depends on both domains and its own ports; `kernel/domain` depends on nothing. No framework, Firebase, or browser coupling exists anywhere in `src` — the only ambient dependencies are `Date`, `RegExp`, and `Object.freeze`. There is no global mutable state, no singleton, and no module-level side effect beyond frozen regex and number constants.

The three published entry points expose a deliberately **primitive-only** boundary: `RegisterUserCommand` is four strings, `execute` returns `Promise<string>`, and all three ports traffic exclusively in `string` and string-literal unions. Value objects are used purely

internally. This is a defensible design — it keeps adapters free of domain-type coupling — but it is undocumented, and it is contradicted by `user-id.types.ts:11-14`, whose JSDoc instructs the reader to "Construct with `Id.of<"User">(value)`", an instruction no consumer can follow.

---

4. Public export and artifact inventory

Artifact: 22,297 bytes packed / 92,772 unpacked / **95 files** (identical file count and unpacked size to published `2.0.0`). Contains `dist/`, `license`, `package.json`, and the 0-byte `readme.md`. No tests, no source, no credentials, no oversized files.

Consumer-visible surface, verified empirically by import probe against the installed tarball:

Specifier	ESM `import`	CJS `require`	Named exports
`.../user/application`	Yes	No — `ERR_PACKAGE_PATH_NOT_EXPORTED`	`RegisterUserUseCase`, `EmailAlreadyRegisteredError`, `UnderageUserError`, `InvalidFullNameError`, `InvalidEmailError`, `InvalidPasswordError`, `InvalidIsoDateError`, type `RegisterUserCommand`
`.../user/ports`	Yes	No	types only: `UserRepositoryPort`, `UserRecord`, `CreateProfileResult`, `AuthProviderPort`, `CreateAccountResult`
`.../kernel/ports`	Yes	No	types only: `HmacIdDeriverPort`
(package root)	No — `No "exports" main defined`	No	—
`.../kernel/domain/value-objects`	No	No	—
`.../dist/**` (deep import)	No	No	—
`.../package.json`	No	No	—

Runtime classes reachable by a consumer: exactly 5 (one use case, four error classes). **Unreachable but shipped: 55 files** — every value object, the `User` entity, and the `kernel/index.ts` and `kernel/domain/index.ts` barrels.

---

5. Scorecard

#	Dimension	Score	One-line basis
1	Purpose, architecture, boundaries	**3 / 5**	Clean hexagonal layering, zero deps, no framework coupling; export map contradicts the stated purpose, no clock port
2	Code quality and correctness	**4 / 5**	Excellent VO discipline and JSDoc, 100% coverage; `Id.of` canonicality inconsistency, non-atomic registration flow
3	Public API design	**2 / 5**	Primitives unreachable, no common error base, bare `string` return, `./package.json` not exported
4	Backward compatibility and semver	**2 / 5**	**Confirmed** breaking change published as a minor; no shipped changelog, no migration docs, no API-diff gate
5	TypeScript and exported types	**4 / 5**	`strict`, accurate declarations, type-level tests, resolves under node16/nodenext/bundler; dangling declaration maps
6	Package artifact and module structure	**3 / 5**	publint-clean, 22 KB, correct conditions; 58% of files unreachable, no `sideEffects`, dangling source maps
7	Runtime and ecosystem compatibility	**2 / 5**	No `engines`, ESM-only undocumented, no CJS, single-Node CI, no compatibility matrix
8	Dependencies and supply chain	**4 / 5**	Zero runtime deps, provenance and Trusted Publishing, SHA-pinned actions, Dependabot; dev-chain CVEs, `prepare` script
9	Security and misuse resistance	**3 / 5**	Strong secret hygiene, no injection surface, no runtime deps; weak password policy, raw input echoed in errors
10	Error contract and diagnostics	**2 / 5**	Named errors and redaction; no base class, no codes, no `cause`, bare `TypeError`, cross-realm `instanceof` fragility
11	Performance and resource management	**4 / 5**	Tiny, synchronous, allocation-light, no I/O or handles; no benchmarks, no `sideEffects` flag
12	Testing strategy	**3 / 5**	422 tests, 100% coverage, type specs, fake timers; no packed-artifact, clean-install, or Node-matrix tests
13	Documentation and consumer experience	**1 / 5**	Empty README, no shipped changelog, no examples, no migration guide; inline JSDoc is the sole bright spot
14	CI/CD and release management	**3 / 5**	Strong PR gates, environment approval, provenance, SHA pins; release skips tests, semver misconfigured, changelog never committed
15	Ownership and maintainability	**2 / 5**	Single maintainer, no CODEOWNERS/CONTRIBUTING/SECURITY, no stated support or EOL policy

Mean: 2.9 / 5

6. Findings by severity

HIGH

H-1 — A breaking public API change was published as a minor release; the release configuration still permits it

Attribute	Value
Dimension	4 — Backward compatibility and semantic versioning
Severity	High
Confidence	High
Status	**Confirmed** — verified against registry tarballs and reproduced from configuration
Effort	Small
Timing	Immediate

Evidence. Comparing the published declaration files for `/user/ports` between `1.5.0` and `1.6.0`, extracted directly from the registry:

```
  export interface UserRepositoryPort {
-     existsById(id: string): Promise<boolean>;
-     save(record: UserRecord): Promise<void>;
+     createProfile(record: UserRecord): Promise<CreateProfileResult>;
  }
  export interface AuthProviderPort {
-     accountExistsById(id: string): Promise<boolean>;
-     createAccount(id: string, email: string, password: string): Promise<void>;
-     updatePassword(id: string, password: string): Promise<void>;
+     createAccount(params: { id; email; password; displayName }): Promise<CreateAccountResult>;
  }
```

Four exported methods removed, one signature and return type replaced — in a **minor** version bump.

Root cause, from `.releaserc.yml:5`: `@semantic-release/commit-analyzer` is used with no `preset`, so it defaults to `angular`, whose header pattern `/^(\w*)(?:\((.*)\))?: (.*)$/` does not admit `!`. Reproduced by invoking the installed analyzer directly:

```
release: null    <= "feat!: refactor application ports (#97)"
release: minor   <= "feat: stabilizing (#99)"
release: major   <= "feat: something\n\nBREAKING CHANGE: ports changed"
--- combined 1.5.0..1.6.0 window --- release: minor
```

The commit `a9d8796` `feat!: refactor application ports (#97)` has **no body** (`git log -1 --format=%b` returns empty). It scored `null`; the release was driven entirely by the unrelated `feat: stabilizing (#99)`. By contrast `3252839` (v2.0.0) carries an explicit `BREAKING CHANGE:` footer and correctly produced a major — so the process only worked by accident of one commit being written differently from the other.

Compounding this: `.commitlintrc.yml:2` extends `@commitlint/config-conventional`, which *does* honour `!`. The repo therefore actively validates and accepts a marker that its release tool silently discards.

Consumer and business impact. Any consumer with `"^1.5.0"` receives `1.6.0` on a routine `npm install` or Dependabot bump, and every adapter implementing `UserRepositoryPort` or `AuthProviderPort` stops compiling. Because these are `interface` types with no runtime presence, a JavaScript consumer would fail at runtime instead — calling `save()` on an object that no longer has it. The second-order effect is worse than the incident: a `feat!:` commit with no footer produces **no release at all**, so an urgent breaking fix can be silently swallowed and the maintainer will believe it shipped.

Failure scenario. A second consumer app pins `^1.5.0`. Dependabot opens a routine minor bump to `1.6.0`. CI type-checks the *app*, which fails opaquely with "Property 'save' does not exist on type 'UserRepositoryPort'" on a PR titled "chore(deps): bump fm-budget-control-core from 1.5.0 to 1.6.0". Nobody suspects a semver breach because minors are assumed safe.

Remediation.

1. Switch both analyzer and notes generator to the preset that honours `!`:

```
- - "@semantic-release/commit-analyzer"
  - preset: conventionalcommits
- - "@semantic-release/release-notes-generator"
  - preset: conventionalcommits
```

(add `conventional-changelog-conventionalcommits` as a devDependency)

2. Add a commitlint rule requiring a `BREAKING CHANGE:` footer whenever `!` is present, so the two tools can never disagree again.
3. Add an API-surface diff gate (see M-10) so a removed export fails CI regardless of commit hygiene.
4. Consider `npm deprecate` on `1.6.0` and `1.5.0` with a message pointing at `2.0.0`, since the version range is misleading in the registry.

Verification. `npx semantic-release --dry-run` on a branch containing a synthetic `feat!:`-only commit must report `major`. Then re-run with only `feat:` and confirm `minor`.

H-2 — The published package contains no documentation of any kind

Attribute	Value
Dimension	13 — Documentation and consumer experience
Severity	High
Confidence	High
Status	Confirmed
Effort	Medium
Timing	Immediate

Evidence. `wc -c readme.md` returns **0**. The empty file is nonetheless packed (`package/readme.md` present in the tarball listing), so npmjs.com renders a blank package page. `files: ["dist"]` at `package.json:7-9` excludes `CHANGELOG.md`, so no changelog reaches consumers either. In git, `CHANGELOG.md` contains only a `1.0.0` section — **duplicated verbatim twice** — while nine versions are published; `@semantic-release/changelog` is configured at `.releaserc.yml:8-9` but `@semantic-release/git` is **not in the plugin list** (despite being a devDependency at `package.json:26`), so each release regenerates the changelog in CI and discards it. There is no `CONTRIBUTING.md`, `SECURITY.md`, or examples directory.

Consumer and business impact. A consumer cannot discover that the root specifier does not resolve, that only three subpaths exist, that the package is ESM-only, that `moduleResolution` must be `node16`, `nodenext`, or `bundler`, or what Node version is required. For a package that has already shipped two breaking rewrites of its port interfaces, there is no upgrade guidance at all — the `1.5.0` to `1.6.0` and `1.6.0` to `2.0.0` migrations exist nowhere in written form. The inline JSDoc is genuinely excellent, but it is invisible until after a successful import.

Failure scenario. A developer adds the dependency, writes `import { Email } from "@fm-budget-control/fm-budget-control-core"`, receives `ERR_PACKAGE_PATH_NOT_EXPORTED`, finds a blank npm page and a changelog frozen at 1.0.0, and abandons the package — or, worse, copies the email regex out of the repo into their own codebase, duplicating the validation this package exists to centralise.

Remediation. Write `readme.md` covering: install, the three subpath entry points with runnable snippets, the ESM-only and `moduleResolution` constraint, the supported Node range, the error taxonomy and how to branch on it, and a migration table for 1.5 to 1.6 to 2.0. Add `readme.md` and `CHANGELOG.md` to `files`. Add `@semantic-release/git` to the `.releaserc.yml` plugin list (assets: `CHANGELOG.md`, `package.json`) so the changelog is committed back.

Verification. `npm pack --dry-run` lists a non-empty `readme.md` and `CHANGELOG.md`; after a dry-run release, `git status` shows a changelog commit; render the README locally to confirm every snippet executes against the packed tarball.

H-3 — Every domain primitive is shipped but unreachable; 55 of 95 published files are dead weight

Attribute	Value
Dimension	3 — Public API design / 6 — Package artifact
Severity	High
Confidence	High
Status	Confirmed
Effort	Small (option B) to Medium (option A)
Timing	Immediate

Evidence. The `exports` map at `package.json:10–23` defines exactly three subpaths and no `"./*"` wildcard, so Node's exports encapsulation blocks everything else. Probed against the installed tarball:

```
FAIL  .../kernel/domain/value-objects              -> ERR_PACKAGE_PATH_NOT_EXPORTED
FAIL  .../dist/kernel/domain/value-objects/index.js -> ERR_PACKAGE_PATH_NOT_EXPORTED
FAIL  .../dist/user/domain/value-objects/index.js   -> ERR_PACKAGE_PATH_NOT_EXPORTED
FAIL  (package root)                               -> No "exports" main defined
```

`Id`, `Money`, `CurrencyCode`, `IsoDate`, `IsoDateTime`, `Email`, `FullName`, `Password`, `User`, `UserId`, and the `kernel/index.ts` and `kernel/domain/index.ts` barrels are all present in the tarball and importable by nobody. The package description is "*Hexagonal architecture **domain primitives***" — the primitives are the one thing a consumer cannot obtain.

The strongest evidence that this is an oversight rather than a deliberate boundary is `user-id.types.ts:11–14`, which documents the intended consumer usage:

> *"Construct with `Id.of<"User">(value)` / `Id.parse<"User">(value)`, never `as UserId`."*

This is an instruction that cannot be carried out from outside the package, because `Id` has no export path.

Consumer and business impact. About 58% of the published bytes are unreachable. More importantly, a consumer needing to validate an email before submitting a form, format a `Money` amount, or compare two `IsoDate` values must reimplement logic that exists, tested to 100%, inside the dependency they already installed. Divergence between the app's validation and the library's is then a matter of time — and the failure mode is a use case rejecting input the UI accepted.

Failure scenario. A web front-end is added to `fm-budget-control` and needs client-side email validation matching the server. It cannot import `Email`, so it copies `LOCAL_PART_REGEX` and `DOMAIN_LABEL_REGEX` from `email.vo.ts:6–7`. Six months later the library tightens domain-label validation in a patch release; the front-end still accepts the old form, and users hit a confusing server-side rejection after submitting a valid-looking form.

Remediation. Decide explicitly, then make the artifact match:

- *If the primitives are meant to be public* — add `"./kernel/domain"` and `"./user/domain"` (or a curated `"./value-objects"`) to `exports`, and treat them as versioned API from then on.
- *If the boundary is deliberately primitives-only* — keep `exports` as-is, but stop shipping the unreachable files (a build that emits only what is exported, or an explicit `files` allowlist), remove the two dead barrels, and correct the `user-id.types.ts` JSDoc so it does not instruct consumers to do the impossible. Also revise the package `description`, which currently promises what the artifact does not deliver.

Independently of that choice, add `"./package.json": "./package.json"` to `exports` — several bundlers and tooling probe it (see L-3).

Verification. Re-run the import probe script against the newly packed tarball; assert every intended specifier resolves and no unintended file ships (`tar -tzf` diff against an expected manifest, ideally as a CI test).

MEDIUM

M-1 — No `engines` field: no Node.js floor is declared to consumers

Dimension 7 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Small · **Timing** Immediate

Evidence. `package.json` has no `engines` key; confirmed absent from published `2.0.0` metadata via `npm view`. The package is `"type": "module"` targeting `ES2022` with `module: NodeNext` (`tsconfig.json:3–5`). `.nvmrc` says `24`, CI pins `node-version: 24` (`quality-gate.yml:35`), and the sole consumer declares `engines.node: "24"` — none of which reaches an installer.

Impact. npm emits no `EBADENGINE` warning on an unsupported runtime. A consumer on Node 16 installs successfully and fails at import with a subpath-exports or syntax error, with nothing pointing at the Node version as the cause.

Failure scenario. A CI image still on Node 16 installs the package; the build fails with `ERR_PACKAGE_PATH_NOT_EXPORTED` on `./user/ports`, and an engineer spends hours before checking `node -v`.

Remediation. Add `"engines": { "node": ">=20" }` (or `>=24` to match what is actually tested), and state the range in the README. Choose the floor deliberately: `ES2022` plus `NodeNext` is satisfied by Node 18+, but nothing below 24 is exercised.

Verification. Install the packed tarball under an out-of-range Node and confirm an `EBADENGINE` warning; add a CI matrix job at the declared floor.

M-2 — `prepare: "husky"` is published in the consumer-facing `package.json`

Dimension 8 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Small · **Timing** Immediate

Evidence. `package.json:37` defines `"prepare": "husky"`, and it is present verbatim in the installed tarball's `package.json`. Installing the tarball into a clean consumer produced:

```
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   @fm-budget-control/fm-budget-control-core@1.0.0 (prepare: husky)
```

`husky` is a devDependency only (`package.json:33`), so it is not present in a consumer's tree.

Impact. Registry tarball installs do not run a dependency's `prepare`, so today's consumers are unaffected — but **git-URL and `file:` installs do**, and would fail with `sh: husky: command not found`. It also trips supply-chain tooling: npm's allow-scripts, `--ignore-scripts` policies, and lockfile-audit gates all flag the package as script-bearing, which in a hardened organisation can block adoption outright.

Failure scenario. A developer pins a pre-release fix with `npm i github:fm-budget-control/fm-budget-control-core#fix-branch`. npm runs `prepare`, `husky` is not installed, and the install aborts with an error that names husky rather than the actual problem.

Remediation. Guard the hook so it is a no-op outside the repo: `"prepare": "husky || true"`, or better, move it out of the published manifest — for example `"prepare": "node -e \"process.env.CI||require('husky')()\""`, or drop `prepare` and initialise husky via a documented one-time `npm run hooks:install`. Also remove or relocate the non-standard `allowScripts` field (`package.json:47–50`), which is published but pins packages (`unrs-resolver`, `fsevents`) that are not even direct dependencies.

Verification. `npm pack --dry-run` then inspect `package.json` in the tarball; install it from a git URL into a scratch project and confirm the install succeeds.

M-3 — Declaration maps and source maps are published, but `src/` is not

Dimension 5, 13 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Small · **Timing** Next

Evidence. `tsconfig.json:11–12` sets `declarationMap: true` and `sourceMap: true`, so 47 `.map` files ship (half the artifact's file count). `files: ["dist"]` excludes `src`. Inspecting the installed package:

```
dist/kernel/domain/value-objects/id/id.vo.d.ts.map -> ['../../../../../src/kernel/domain/value-objects/id/id.vo.ts']
dist/kernel/application/ports/index.js.map         -> sourcesContent: NO
$ ls src  ->  No such file or directory
```

Every map points at a path that does not exist in the tarball, and no `sourcesContent` is inlined.

Impact. "Go to Definition" in a consumer's editor navigates to a nonexistent `.ts` file instead of falling back to the readable `.d.ts` — strictly worse than shipping no declaration maps. Stack traces from consumer debugging sessions resolve to missing sources. The maps are roughly a third of the unpacked size for negative value.

Failure scenario. A consumer developer clicks through to `RegisterUserUseCase` expecting to read its contract, and the editor reports the file cannot be opened — the excellent JSDoc in `register-user.use-case.ts` is never seen.

Remediation. Pick one: (a) add `"src"` to `files` so the maps resolve and consumers get true source navigation; (b) set `"declarationMap": false, "sourceMap": false` in `tsconfig.build.json` and drop the 47 map files; or (c) keep maps and add `"inlineSources": true` so content travels with them. Option (a) also gives consumers readable source, which partly mitigates H-2.

Verification. Install the packed tarball into a scratch TS project, click through to an exported symbol, and confirm it opens real content.

M-4 — The use case reads the ambient system clock; every other dependency is a port

Dimension 1, 2, 12 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Medium · **Timing** Next

Evidence. `register-user.use-case.ts:33` — `const now = IsoDateTime.of(new Date().toISOString());` — and `user.entity.ts:89` — `const today = new Date();`. The class already injects three ports (`UserRepositoryPort`, `AuthProviderPort`, `HmacIdDeriverPort`); time is the one dependency that bypasses the pattern. The entity's own test compensates with `jest.useFakeTimers()` and `jest.setSystemTime()` (`user.entity.spec.ts:11-12`), while `register-user.use-case.spec.ts` runs against the real clock with a 1990 birth date that happens to stay valid.

Impact. Consumers cannot control `createdAt` / `updatedAt` or the age-check reference date, so any consumer test touching registration must mutate global time — which is process-wide and interacts badly with parallel test runners. It also blocks legitimate use cases: backfilling historical records, or replaying events with their original timestamps.

Failure scenario. A data-migration script imports legacy users through `RegisterUserUseCase` to reuse its validation. Every imported record is stamped with today's date, silently destroying the original `createdAt` — and a user who was 17 at their original signup date and is now 18 is silently admitted, inverting the business rule.

Remediation. Add a `ClockPort { now(): IsoDateTime }` to `kernel/application/ports`, inject it into `RegisterUserUseCase`, and pass an explicit `today: IsoDate` (or the clock) into `User.create`. Note this changes the constructor arity — ship it in a major, with the `!` plus footer convention from H-1.

Verification. A use-case test that injects a fixed clock and asserts exact `createdAt` / `updatedAt` values, with no `jest.useFakeTimers()` anywhere in the suite.

M-5 — The error contract has no common base class and no machine-readable codes

Dimension 10, 3 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Medium · **Timing** Next

Evidence. Every validation error extends `TypeError` — `id.vo.ts:3`, `email.vo.ts:9`, `password.vo.ts:6`, `money.vo.ts:3`, `underage-user.error.ts:1` — while `register-user.errors.ts:1` extends `Error`. There is no shared `DomainError` base, no `code` property, and no `cause` propagation anywhere. `RegisterUserUseCase` additionally throws a **bare, unnamed** `TypeError` at `register-user.use-case.ts:26-28` for a deriver contract violation.

The cost is visible in the real consumer, which must enumerate five classes by hand:

```
// register-user.function.ts:72-78
if (error instanceof InvalidFullNameError || error instanceof InvalidEmailError ||
    error instanceof InvalidPasswordError || error instanceof InvalidIsoDateError ||
    error instanceof UnderageUserError) {
  return new HttpsError("invalid-argument", error.message);
}
```

Impact. Two concrete problems. First, adding any new validation error to the package silently degrades it to `HttpsError("internal", ...)` in every existing consumer — a validation failure presented to the user as a server fault, with no compile-time signal. Second, `instanceof` across a package boundary is realm-fragile: if npm ever installs two copies of the package (a transitive range conflict), every one of these checks silently evaluates false.

Failure scenario. A `MaxBudgetExceededError` is added in a minor release. The Firebase function maps it to `"internal"`, the client shows "Registration could not be completed. Please try again.", and the user retries forever against a deterministic validation failure. Nothing in CI catches it, because the package's own tests pass and the consumer's types still compile.

Remediation. Introduce an exported `DomainError extends Error` base with a readonly `code: string` (and `name`), have every domain error extend it, and export the base plus an `isDomainError(e): e is DomainError` predicate that checks the `code` property rather than the prototype chain. Replace the bare `TypeError` with a named `InvalidDerivedIdError`. Preserve `cause`

when wrapping adapter failures. Document the code list as versioned API. Reconsider `extends TypeError` — a rejected email is a validation failure, not a type error, and `instanceof TypeError` catch blocks elsewhere will swallow them.

Verification. A consumer-facing test asserting `isDomainError(err) && err.code === "INVALID_EMAIL"` across all exported errors, plus a test that the predicate works on an object from a *different* module instance.

M-6 — Registration is non-atomic across two ports, with no compensation or documented idempotency contract

Dimension 2, 3 · **Severity** Medium · **Confidence** Medium · **Confirmed (behaviour) / Probable (impact)** · **Effort** Medium · **Timing** Next

Evidence. `register-user.use-case.ts:49-70` calls `authProvider.createAccount(...)` first, then `userRepository.createProfile(...)`, then throws `EmailAlreadyRegisteredError` if the profile already exists. There is no `try` / `catch`, no rollback, and no `finally`. The inline comment at lines 44-48 acknowledges the "previous attempt that died before creating the profile" case and handles it by adopting the existing auth uid — good — but the reverse ordering means an auth account is always created before the profile is attempted.

Impact. If `createProfile` throws (network partition, Firestore quota, permission error), an auth account exists with no corresponding profile. The design partially self-heals: a retry gets `status: "email-already-exists"`, adopts the canonical uid, and creates the profile. But if the caller never retries, the orphaned auth account persists indefinitely — it can sign in, and every downstream lookup of its profile returns nothing. The `EmailAlreadyRegisteredError` path also leaves the just-created auth account in place by design, which is defensible but nowhere documented as part of the port contract.

Failure scenario. Firestore has a brief outage during a signup spike. Fifty users get auth accounts and no profiles. They can authenticate, land in the app, and every profile-dependent screen errors. Support cannot distinguish them from genuine bugs because nothing recorded the partial state.

Remediation. Document the exact ordering, retry semantics, and idempotency guarantee in the `AuthProviderPort` and `UserRepositoryPort` JSDoc — this is a port *contract*, and adapter authors currently have to infer it from the use case body. Then either (a) wrap the `createProfile` failure so the caller can distinguish "auth created, profile failed" from a total failure, enabling a targeted retry, or (b) reverse the order so the profile is written first under the derived id and the auth account is created last, making the derived id the sole source of truth. Option (b) conflicts with the v2.0.0 canonical-uid decision, so (a) is likely the pragmatic path.

Verification. A test where `createProfile` rejects, asserting the thrown error carries enough context to identify the orphaned uid; plus an integration test that re-running `execute` after such a failure converges to a complete, consistent state.

M-7 — The release job skips lint, type-check, and tests, and publishes an artifact no gate has inspected

Dimension 14 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Small · **Timing** Next

Evidence. `release.yml:47-52` runs only `npm run build` before `npx semantic-release`, with the rationale:

> "*Lint, type-check, and tests are intentionally omitted here. workflow_dispatch can only be triggered on main, which has already passed the full CI gate via PR.*"

The reasoning is sound for code identity, but the release job re-installs and rebuilds in a fresh runner, so the bytes that reach npm were produced by a job whose output nothing verified beyond `publint` (which runs inside `build`). There is no `semantic-release --dry-run` preview step, and no inspection or diff of the packed tarball before publish.

Impact. Any divergence introduced by the release-time install or build — a transitive devDependency resolving differently, a `tsc` behaviour change, an accidental file landing in `dist` — reaches the registry unreviewed. npm publishes are effectively irreversible (unpublish is restricted to a 72-hour window, and republishing a version is forbidden), so a bad artifact must be fixed forward under a new version.

Failure scenario. A `typescript ^6.0.3` patch changes emit in a way that breaks a declaration. Main's CI ran against the previous patch; the release job installs the new one, builds, `publint` passes (it checks packaging, not semantics), and a subtly broken `.d.ts` ships as a patch release.

Remediation. Add `npm run typecheck && npm run test && npm run test:types` to the release job — the latency cost is under two minutes against an irreversible publish. Add an `npx semantic-release --dry-run` step that prints the computed version and

release notes before the real run. Add a tarball manifest assertion (`tar -tzf` compared to a checked-in expected file list) so unexpected additions or removals fail the release rather than shipping.

Verification. Trigger a `workflow_dispatch` on a no-op commit and confirm the dry-run reports "no release", with all gates green.

M-8 — The password policy caps length at 16 characters and requires no letter-case variety

Dimension 9 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Small (code) / Medium (with migration) · **Timing** Next

Evidence. `password.vo.ts:1-4`: `MIN_LENGTH = 5`, `MAX_LENGTH = 16`, requiring only one digit and one special character. Enforced at `password.vo.ts:65-73`.

Impact. A 16-character ceiling is the more serious half. It rejects password-manager-generated credentials (commonly 20-32 characters) and passphrases — the two things that most improve real-world account security — while a 5-character floor admits credentials like `a1!bc`. NIST SP 800-63B recommends a minimum of 8 and requires that verifiers permit **at least 64** characters, and explicitly advises against composition rules of the digit-plus-symbol kind used here. Because this VO is the sole gate before `AuthProviderPort.createAccount`, it is the effective account-security policy for the whole product.

Failure scenario. A security-conscious user pastes a 32-character generated password. Registration fails with "Password must be between 5 and 16 characters", surfaced verbatim to the client via `HttpsError("invalid-argument", error.message)`. The user picks something short and memorable instead, and the product's aggregate credential strength is lower than if no policy existed.

Remediation. Raise `MAX_LENGTH` to at least 64 (128 is a common choice; note bcrypt truncates at 72 bytes if the adapter uses it — confirm what the auth provider does before choosing). Raise `MIN_LENGTH` to 8. Consider dropping the composition rules in favour of a breached-password check at the adapter layer. Because raising the maximum narrows nothing and widens acceptance, it is a **minor**-compatible change; raising the minimum is breaking for existing weak credentials and belongs in a major with a documented migration.

Verification. Extend `password.vo.spec.ts` with boundary cases at 8, 64, and 65 characters, and confirm end-to-end that the auth adapter accepts the full range.

M-9 — Critical and high dev-dependency advisories sit inside the privileged release job

Dimension 8 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Small (audit fix) / Medium (job split) · **Timing** Next

Evidence. `npm audit --omit=dev` returns **0 vulnerabilities** (the package has zero runtime dependencies). `npm audit` returns **7 vulnerabilities (1 critical, 5 high, 1 moderate)**, all dev-only:

Severity	Package	Path
Critical	`tar`	node-tar: PAX numeric path type confusion; unbounded decompression DoS
High	`undici`	via `@semantic-release/github@12.0.8`, `@semantic-release/npm@13.1.5` to `@actions/http-client`, `npm` to `node-gyp`
High	`ip-address`	Address4 octal/decimal confusion, leading to SSRF and trust-boundary bypass
High	`fast-uri`, `js-yaml`, `brace-expansion`	host confusion; quadratic-complexity DoS

Impact. No consumer is exposed — none of this ships. But these packages execute in the `release` job, which holds `contents: write`, `issues: write`, `pull-requests: write`, and `id-token: write` (npm publishing rights via OIDC) — the highest-privilege context in the repository. `tar` and `ip-address` in particular process remote-controlled input (registry tarballs, API responses) inside that context.

Failure scenario. A compromised or malicious transitive dependency in the semantic-release tree exploits one of these during a release run, obtains the short-lived OIDC token, and publishes a trojaned version of the package under valid provenance. The provenance attestation would be genuine, making detection substantially harder.

Remediation. Run `npm audit fix` and keep the tree current (Dependabot is configured and grouping dev updates — verify those PRs are actually being merged, since several advisories predate the last dev bump). Add `npm audit --audit-level=high` as a **non-blocking** CI step so drift is visible. Consider splitting the release into a build job (no privileges, uploads an artifact) and a minimal publish job (privileges, downloads the artifact) so the large dev tree never runs alongside the OIDC token.

Verification. `npm audit` reports no critical or high findings after the fix; confirm the release job's dependency tree by inspecting its `npm ci` output.

M-10 — No automated public-API compatibility checking

Dimension 4, 14 · **Severity** Medium · **Confidence** High · **Confirmed** · **Effort** Medium · **Timing** Next

Evidence. The quality gate runs lint, typecheck, type-specs, tests, coverage, Sonar, and build. `publint` runs inside `build` — it validates *packaging* (exports shape, file presence, module conditions) and correctly reports "All good!", but it has no notion of API surface. Nothing compares the exported types of `HEAD` against the last published version. This is precisely the gap that let H-1 reach the registry unnoticed.

Impact. Removing an exported symbol, narrowing a parameter, or widening a return type produces zero CI signal. The only defence is commit-message discipline, which H-1 proves is insufficient.

Remediation. Add an API-surface gate. Two complementary options:

- `@microsoft/api-extractor` with a committed `.api.md` report — a PR that changes the public surface must update the report, making the change visible in review.
- A CI step that packs `HEAD`, packs the current `latest` from npm, and diffs the `.d.ts` files, failing on any removal unless the PR is labelled `breaking`.

Also add `@arethetypeswrong/cli` (`attw --pack`), which catches types-versus-runtime resolution mismatches across module modes that `publint` does not.

Verification. Open a scratch PR that deletes an exported symbol and confirm CI fails.

LOW

ID	Title	Dimension	Confidence	Evidence and impact	Remediation	Effort
L-1	`Id.of` accepts non-canonical whitespace, so `of` and `parse` can yield unequal `Id`s for the same input	2, 3	High · Confirmed	`id.vo.ts:54–56` validates `value.trim().length > 0` but `id.vo.ts:26–32` stores the untrimmed value. `Id.of(" x ")` succeeds and is **not** `.equals(Id.parse(" x "))`. Tested as intended behaviour (`id.vo.spec.ts:26–32`), but it contradicts the `of` -means-canonical contract that `iso-date.vo.ts:81–85` and `full-name.vo.ts:83–88` both document explicitly. Two logically identical ids compare unequal, producing duplicate records	Make `Id.isValid` reject any value where `value !== value.trim()`, matching the sibling VOs; update the spec	Small
L-2	ESM-only with no CJS condition; legacy `moduleResolution: node` unsupported and undocumented	7	High · Confirmed	`require()` of all three subpaths fails with `ERR_PACKAGE_PATH_NOT_EXPORTED`. TS resolves cleanly under `nodenext`, `node16`, and `bundler` but fails under `node` with TS2307 (reproduced). Defensible for a Node-24 Firebase target, but a consumer discovers it only by hitting it	Document the constraint prominently in the README; add `"require"` conditions only if a CJS consumer actually materialises	Small
L-3	`./package.json` is not exported	6	High · Confirmed	`import ".../package.json"` fails with `ERR_PACKAGE_PATH_NOT_EXPORTED`. Some bundlers, `attw`, and version-probing tools read it	Add `"./package.json": "./package.json"` to `exports`	Small
L-4	No `sideEffects` field	6, 11	High · Confirmed	Absent from `package.json` and published metadata. All modules are genuinely side-effect-free (top-level regex and number consts only), so bundlers are being denied a safe optimisation. Impact is small for a Node consumer	Add `"sideEffects": false`	Small
L-5	Validation errors echo raw user input, which the consumer reflects to the client and logs	9, 10	Medium · Confirmed	`Invalid email address: "${value}"` (`email.vo.ts:11`), same pattern in `InvalidFullNameError`, `InvalidIsoDateError`, `InvalidIdError`. The consumer passes `error.message` straight into `HttpsError("invalid-argument", ...)` (`register-user.function.ts:79`). It is the caller's own input, so disclosure risk is minimal — but it lands in logs verbatim, and unbounded input length is echoed. `Password` correctly does **not** do this (`password.vo.ts:79–93`)	Truncate echoed values (for example 64 chars) or move the raw value to a non-message property so consumers opt in deliberately	Small
L-6	Inconsistent error taxonomy: validation errors extend `TypeError`, `EmailAlreadyRegisteredError` extends `Error`	10	High · Confirmed	See M-5. A generic `catch (e) { if (e instanceof TypeError) }` in consumer code swallows domain validation failures as programming errors	Fold into the `DomainError` base introduced in M-5	Small
L-7	Release workflow comment references the wrong package path	14	High · Confirmed	`release.yml:63` points at `npmjs.com/package/@fm-budget-control/budget-core/access` — the package is `fm-budget-control-`	Correct the path	Small

				`core`. Comment-only, but it is the operator's pointer to the Trusted Publisher config during an incident		
L-8	Non-standard `allowScripts` field published	8	High · Confirmed	`package.json:47–50` ships to consumers and pins `unrs-resolver@1.12.2` and `fsevents@2.3.3`, neither a direct dependency. Harmless noise in the public manifest	Move to a local config or remove	Small
L-9	`package.json` `version` is permanently stale at `1.0.0` in git	4, 15	High · Confirmed	Registry `latest` is `2.0.0`; every local script logs `@...core@1.0.0`. `@semantic-release/git` is a devDependency but absent from the `.releaserc.yml` plugin list, so neither version nor changelog is committed back. A checkout tells you nothing about what is released	Add `@semantic-release/git` with assets `["CHANGELOG.md", "package.json"]` (also fixes half of H-2)	Small
L-10	No CODEOWNERS, CONTRIBUTING, or SECURITY.md; single maintainer	15	High · Confirmed	All three absent. `npm view` shows one maintainer (`fabiomoggi`). No documented vulnerability-reporting channel, review requirement, support expectation, or EOL policy for a **publicly published** package	Add SECURITY.md with a disclosure contact (highest priority of the three, given public publication), CODEOWNERS, and a short CONTRIBUTING covering the commit convention from H-1	Small
L-11	`execute` returns a bare `string`; deriver contract violation throws an unnamed `TypeError`	3, 10	High · Confirmed	`register-user.use-case.ts:17` returns `Promise<string>` — the caller cannot tell whether the account was newly created or adopted from an existing auth uid, information the use case has and discards (`register-user.use-case.ts:49–54`). Line 26 throws a bare `TypeError`, indistinguishable from a genuine programming error	Return `{ userId: string; status: "created" or "adopted" }`; replace the bare throw with a named error (breaking, so major)	Small
L-12	Two barrel modules are unreachable from the export map	6	High · Confirmed	`src/kernel/index.ts` and `src/kernel/domain/index.ts` are compiled and shipped but resolve to no export path. `src/kernel/domain/index.ts` also lacks a trailing newline and semicolon	Remove, or expose — see H-3	Small

7. Positive practices, with evidence

These are real strengths and should be preserved through any remediation.

- **Zero runtime dependencies.** `npm audit --omit=dev` reports *found 0 vulnerabilities*; `dependencies`, `peerDependencies`, and `optionalDependencies` are all empty. The consumer's entire supply-chain exposure from this package is the package itself. This is the single best property of the artifact.
- **npm Trusted Publishing with verified provenance.** `release.yml:21` grants `id-token: write` with no long-lived npm token; `npm view` confirms a live SLSA v1 provenance attestation and registry signature on `2.0.0`.
- **Release gated behind a manual approval environment.** `release.yml:30` sets `environment: production`, `workflow_dispatch`-only with a mandatory reason input, and `cancel-in-progress: false` so a release is never interrupted mid-publish.
- **All GitHub Actions pinned to full commit SHAs** with version comments (`quality-gate.yml:30`, `ci.yml:36–39`), plus `actions/dependency-review-action` on every PR. Dependabot covers both npm and Actions.

- **Workflow injection correctly avoided.** `issue-branch.yml` handles attacker-controllable issue titles through `actions/github-script` and `process.env` (lines 127-131, 152-155) rather than shell interpolation — the common mistake, avoided.
- **422 tests, 100% statements / branches / functions / lines** across all ten instrumented modules, running in 0.9s. Tests exercise public behaviour and edge cases (whitespace, Unicode names, leap years, `-0` normalisation, currency mismatch), not implementation details.
- **Type-level testing** with a dedicated tsconfig (`tsconfig.type-tests.json`, `id.vo.type-spec.ts`) — rare and valuable for a types-first package, and it validates the phantom-brand mechanism that ordinary tests cannot reach.
- **Deliberate secret hygiene on `Password`.** The raw value is a private field, `toString()` and `toJSON()` both return `"********"` (`password.vo.ts:119-125`), plaintext is reachable only via the pointedly named `revealForHashing()`, and the error message describes the violated *rule* without echoing the value (`password.vo.ts:79-93`). A password cannot be leaked by an accidental `console.log` or `JSON.stringify`.
- **Correct, timezone-safe date handling.** `IsoDate.isValid` explicitly defeats JS `Date` auto-correction (`iso-date.vo.ts:99-106`) so `2024-02-31` is rejected rather than silently becoming March 2; all comparisons route through UTC accessors, with the reasoning documented in the class JSDoc. `IsoDateTime` accepts only `Z`-suffixed UTC. The Feb-29 birthday clamp in `user.entity.ts:95-100` is a genuinely subtle case, handled and tested.
- **Sound `Money` modelling.** Integer minor units with `Number.isSafeInteger` validation (`money.vo.ts:148-150`), so overflow throws rather than silently losing precision; `-0` normalisation (`money.vo.ts:152-154`); a currency-mismatch guard on every operation; and an explicit refusal to encode currency decimal places, correctly deferred to the owning module.
- **Consistent, well-reasoned VO factory triad** (`of` strict / `parse` normalising / `tryParse` non-throwing) with the contract documented on each `isValid`, including explicit warnings against future modification (`iso-date.vo.ts:81-85`, `full-name.vo.ts:83-88`). Where the triad does not apply, the omission is justified in prose (`password.vo.ts:33-39`).
- **Immutability throughout** — every VO and the `User` entity call `Object.freeze(this)` in a private constructor; `User.updateName` and `User.updateEmail` return new instances.
- **Unicode-aware name validation** using `\p{L}\p{M}` with the `u` flag (`full-name.vo.ts:9`), correctly handling "José da Silva", "Anne-Marie", and "D'Angelo" rather than the ASCII-only regex most codebases ship.
- **`publint` wired into `build`** (`package.json:31`), so packaging correctness is enforced at every build, not just at release.
- **JSDoc that explains *why*.** `id.vo.ts:10-18`, `user-id.types.ts:3-15`, and `register-user.use-case.ts:44-58` document rationale and pitfalls, not restated signatures. This is the package's real documentation — which is exactly why H-2 and M-3 are costly, since consumers currently cannot see it.
- **Tiny artifact** — 22 KB packed, no I/O, no timers, no listeners, no caches, no unbounded collections, nothing to leak.

---

8. Compatibility and testing gaps

Gap	Current state	Risk
Node version matrix	Single version (24) in CI; no `engines`	An unsupported-runtime failure is discovered by a consumer, not by CI (M-1)
Packed-artifact test	None — all tests run against `src` via ts-jest	The thing that ships is never imported by a test; H-3 would have been caught instantly by one
Clean-install test	None	No verification that a fresh consumer can install and import
ESM / CJS import test	None	The `require()` failure reproduced in this assessment is untested and undocumented
TypeScript version matrix	Built with TS 6.x; consumer on TS 5.9 (works, verified)	A declaration-emit incompatibility with older TS would surface downstream
`moduleResolution` coverage	None	Legacy `node` resolution fails (reproduced); no CI signal
API-surface regression	None (M-10)	Directly caused H-1
Barrel / entry-point coverage	The three exported barrels appear in no coverage report	Export wiring is entirely untested
Consumer example tests	No examples exist (H-2)	Nothing keeps documentation honest
Bundler / browser	Not applicable under the stated assumption, but unstated	An unverified assumption
Mutation testing	None	100% line coverage does not imply assertion strength

The most valuable single addition is an **artifact-level test**: pack, install into a temp directory, then import every declared entry point, assert the exported symbol names, and `tsc` a consumer file under `nodenext`. That one job would have caught H-3, L-2, and L-3, and would guard against H-1 regressions on removed exports.

---

9. Prioritised remediation roadmap

Immediate — before the next publish

1. **H-1** — switch to the `conventionalcommits` preset; add a commitlint rule tying `!` to a `BREAKING CHANGE:` footer. *(Small)*
2. **H-2** — write `readme.md`; add `readme.md` and `CHANGELOG.md` to `files`; add `@semantic-release/git` to the plugin list (also resolves L-9). *(Medium)*
3. **H-3** — decide the domain-primitive boundary and make `exports` and the shipped file set agree; fix the misleading `user-id.types.ts` JSDoc; add `"./package.json"` (L-3). *(Small to Medium)*
4. **M-1** — add `engines.node`. *(Small)*
5. **M-2** — neutralise the published `prepare` script; drop `allowScripts` (L-8). *(Small)*
6. **L-10** — add `SECURITY.md` with a disclosure contact. *(Small)*

Next — within the following one or two releases

7. **M-10** — API-surface gate (`api-extractor` report plus `attw --pack`) and the packed-artifact / clean-install test from section 8. *(Medium)*
8. **M-7** — restore test gates to the release job; add `--dry-run` preview and a tarball manifest assertion. *(Small)*
9. **M-9** — `npm audit fix`; add a non-blocking audit step; consider splitting build from publish. *(Small to Medium)*
10. **M-5, L-6, L-11** — `DomainError` base with `code`, an `isDomainError` predicate, a named deriver error, and a structured `execute` return. Bundle as one **major**. *(Medium)*
11. **M-3** — resolve the dangling maps (ship `src`, or disable maps, or inline sources). *(Small)*
12. **M-8** — raise the password maximum to 64 or more now (non-breaking); schedule the minimum increase for a major. *(Small)*
13. **M-4** — introduce `ClockPort`; ship in the same major as item 10. *(Medium)*

14. **L-1** — align `Id.of` with the canonical-form contract (breaking; same major). *(Small)*

Later

15. **M-6** — document port idempotency and ordering contracts; add failure-path context so orphaned auth accounts are diagnosable. *(Medium)*
16. Node version matrix in CI at the declared `engines` floor; TypeScript version matrix. *(Medium)*
17. **L-4** `sideEffects: false`; **L-5** truncate echoed input; **L-7** fix the comment; **L-12** remove dead barrels. *(Small each)*
18. Governance: CODEOWNERS, CONTRIBUTING, a written support and EOL policy; address the single-maintainer bus factor (L-10). *(Medium)*
19. Consider mutation testing to validate the assertion strength behind the 100% coverage figure. *(Medium)*

---

10. Open questions that could materially affect conclusions

1. **Is the primitives-only boundary deliberate?** This determines whether H-3 is a packaging bug or a documentation bug, and changes its remediation from "expose the VOs" to "stop shipping them". The `user-id.types.ts` JSDoc suggests the former; the export map suggests the latter.
2. **Are there consumers beyond the Firebase Functions backend?** Exactly one was found, pinned at `^2.0.0` and unaffected by H-1. If the package is public because it may be reused elsewhere, H-1, H-2, and M-1 all rise in severity; if it is public only incidentally, making it private or restricted would retire several findings at a stroke.
3. **Is `main` protected, and are PR reviews required?** Branch and tag protection rules could not be read. The release job's rationale for skipping tests (M-7) rests entirely on "main has already passed the full CI gate via PR" — which only holds if direct pushes to `main` are blocked.
4. **Who is on the `production` environment's required-reviewer list?** If it is the sole maintainer, the approval gate is a speed bump, not a control (L-10).
5. **What is the intended Node floor?** `.nvmrc`, CI, and the consumer all say 24, but the emitted code would run on 18. This decides the `engines` value in M-1.
6. **Does the auth adapter use bcrypt?** If so, its 72-byte truncation caps the useful password length and should inform the M-8 maximum.
7. **Was the `1.5.0` to `1.6.0` break noticed and absorbed at the time?** If the consumer was updated in lockstep, the incident had no impact — but the configuration defect that caused it is still live, so the finding stands regardless.
8. **Is npm 2FA enforced on the maintainer account, and is the Trusted Publisher scoped to the `release` workflow on `main` only?** Not verifiable without registry admin access; both materially affect the supply-chain posture behind M-9.
9. **Is there a documented rollback procedure for a bad publish?** None was found. Given npm's 72-hour unpublish window, the absence of a written procedure is itself a release-management risk.

Final rating

NOT READY

The package is well-engineered internally and genuinely secure in its handling of secrets, dates, money, and dependencies — the zero-dependency, provenance-signed, 100%-covered core is better than most libraries of its size. But as an **independently maintained product** it fails on the two things a consumer depends on most: a trustworthy version contract and any documentation at all. The semver breach is not hypothetical — it was confirmed in the registry and its root cause reproduced in the release configuration, where it remains unfixed.

Minimum conditions to reach "Ready with accepted risks"

1. **H-1 fixed and proven** — `conventionalcommits` preset configured, commitlint rule tying `!` to a `BREAKING CHANGE:` footer, and a `--dry-run` demonstrating that a `feat!:` commit yields `major`.

2. **H-2 fixed** — a non-empty README shipped in the tarball covering entry points, ESM and `moduleResolution` constraints, supported Node, the error taxonomy, and 1.5 to 1.6 to 2.0 migration; changelog committed and published.
3. **H-3 resolved either way** — the export map and the shipped file set agree with a stated, documented boundary.
4. **M-1 and M-2 fixed** — `engines.node` declared; the `prepare` script no longer able to break a consumer install.
5. **M-10 in place** — an API-surface gate plus a packed-artifact import test in CI, so H-1 cannot recur through commit-message discipline alone.

With those five done, the remaining Medium findings (M-3 through M-9) are reasonable to accept explicitly and schedule, since none of them can silently break a consumer.

To reach "Ready"

Additionally: M-5, M-4, and L-11 shipped as a coherent major (error base class with machine-readable codes, injected clock, structured return); test gates restored to the release job (M-7); a Node version matrix at the declared floor; and `SECURITY.md` plus a written support and EOL policy for a publicly published package.